

VELLORE INSTITUTE OF TECHNOLOGY
CHENNAI

School of Computer Science Engineering and Information Systems
Academic Year 2026 – 2027

BCSE408L — CLOUD COMPUTING

Cloud-Native SaaS Project

Review 2 and Final Demonstration (Review 3) — Combined Submission

ServiceDesk AI

A Cloud-Native Customer Support SaaS Platform for Small Businesses

Project 19 · Customer Support SaaS

“Manage customer complaints and automate ticket handling.”

LIVE APPLICATION

<https://servicedesk-ai-two.vercel.app>

API <https://servicedesk-api-ss2d.onrender.com/api> · <https://github.com/ItzGOLLY/servicedesk-ai>

SUBMITTED BY

Name	Registration Number	Programme
Aarush Jagannathan	23BLC1211	B.Tech. Electronics and Computer Engineering
Gomatheswar M	23BLC1195	B.Tech. Electronics and Computer Engineering

UNDER THE GUIDANCE OF

Dr. M Dinakaran

School of Computer Science Engineering and Information Systems

SEPTEMBER 2026

Part A — Deliverables Map for Review 2 and the Final Demonstration

This submission covers Review 2 and the Final Demonstration (Review 3) together. Every deliverable named in the official Review Schedule (Attachment 2) is listed below with the section of this document that satisfies it, the evidence that it is complete, and the point in the live demonstration at which it is shown. Nothing in this document is planned or assumed: every status was verified against the running application on 19 September 2026.

A.1 Live endpoints

What	Address	Verified
Live application (frontend, Vercel CDN)	https://servicedesk-ai-two.vercel.app	✓ Sign-in and all three role dashboards
REST API (Render, Singapore)	https://servicedesk-api-ss2d.onrender.com/api	✓ /health returns database: connected
Interactive API documentation (Swagger UI)	https://servicedesk-api-ss2d.onrender.com/api/docs	✓ 13 documented paths generated from Zod schemas
Cloud database	Render managed PostgreSQL 16 + pgvector (internal network only)	✓ 5 migrations applied on deploy
WhatsApp channel	Meta WhatsApp Cloud API test number +1 (555) 148-1137	✓ Inbound → ticket → auto-reply in ~1 s; agent reply delivered to phone
Source repository	https://github.com/ItzGOLLY/servicedesk-ai	✓ Public, CI green on main

A.2 Review 2 deliverables

Deliverable	Where in this document	Evidence / demo point
System Architecture	Section 11	Figure 11.1; explained in demo step 1
Cloud Architecture	Section 12	Figure 12.1; Render + Vercel dashboards shown in demo step 9
Database Design	Section 13	Nine tables, constraints, indexes, migrations; live data on the admin dashboard
ER Diagram	Section 13.2, Figure 13.1	Derived directly from migration 001–003
Module Design	Section 14	Five specified modules plus WhatsApp, AI and knowledge base
API Design	Section 16.2 and Appendix B	Swagger UI live at /api/docs — demo step 8
Authentication	Section 16.3	Register, sign in, refresh, deactivation — demo step 2
Core Modules	Sections 14 and 16	Ticket lifecycle end to end — demo steps 3–7

Deliverable	Where in this document	Evidence / demo point
GitHub Repository	Section 16.1	Public repository, meaningful history, README, CI badge — demo step 10
Initial Cloud Deployment	Section 18	Live URL opened first in the demo

A.3 Final Demonstration deliverables

Deliverable	Status	Where / how it is shown
Complete SaaS Application	✓	All ten functional requirements implemented and verified — Sections 9 and 19
Public Live URL	✓	https://servicedesk-ai-two.vercel.app
Responsive UI	✓	Verified at 390, 768 and 1366 px — Section 15, Figure 15.12; phone view in the demo
Cloud Database	✓	Render managed PostgreSQL 16 with pgvector — Sections 12 and 13
Testing	✓	161 automated tests, all passing on 19 September 2026; CI on every push — Section 17
Final Report	✓	This document follows the 22-section template from Attachment 2
PPT	✓	docs/ServiceDesk_AI_Final_Demo.pptx in the repository
GitHub Repository	✓	https://github.com/ltzGOLLY/servicedesk-ai
Working Demonstration	✓	Script in Appendix A — 8 minutes, rehearsed against the live system

A.4 Final demonstration checklist (Attachment 2)

Checklist item	Demo step	What the panel sees
Registration / Login	2	A new customer registers and lands on their dashboard; admin signs in with a different role
Role-based Access	5, 6	Customer sees only own tickets; agent sees all; admin-only menu; direct API call by the wrong role returns 403
Core Workflow	3–7	Complaint arrives on WhatsApp → ticket auto-classified → agent drafts with AI → reply reaches the phone → status changes → customer closes
Cloud Deployment	1, 9	Live URL; Render and Vercel dashboards; render.yaml Blueprint
Live URL	1	https://servicedesk-ai-two.vercel.app
GitHub Repository	10	Commit history, README, green CI run, tests folder
Responsive Design	11	Same page on a phone-width viewport: sidebar collapses, tables become cards
Question & Answer by both team members	—	Both members prepared from docs/VIVA_PREPARATION.md and docs/PROJECT_DEEP_DIVE.md

A.5 Mandatory technical expectations (Attachment 1, §3)

Requirement	Status	Implementation
User Registration & Login	✓	JWT access token (15 min, in memory) + httpOnly refresh cookie (7 days); bcrypt cost 12
Role-Based Access	✓	Customer / Agent / Admin; enforced by route middleware and per-record ownership checks in the API
CRUD Operations	✓	Tickets, messages, users, categories, knowledge-base articles
Dashboard & Reports	✓	Three role dashboards, seven reports, CSV export — all computed from the database on each request
Cloud Database	✓	Render managed PostgreSQL 16 with the pgvector extension
REST APIs	✓	~64 endpoints across 11 groups; OpenAPI 3 specification and Swagger UI
Responsive UI	✓	Tailwind design system, responsive from 375 px
Live Cloud Deployment	✓	Vercel (frontend) and Render (API + database) via Blueprint
GitHub Repository	✓	One team repository, regular commits, README, CI

A.6 Demonstration accounts

Role	Email	Password
Admin	admin@servicedesk.ai	Admin@12345
Support Agent	priya.agent@servicedesk.ai	Agent@12345
Customer	rohan@example.com	Customer@12345

These are seeded demonstration credentials. The admin password will be changed immediately after the evaluation.

2. Abstract

A small business that receives customer complaints across email, messaging applications and telephone calls has no reliable record of what was reported, who is handling it, or whether it was ever resolved. Requests are lost in shared inboxes, the same issue is answered twice by different people, and the business cannot measure how long anything took. Commercial helpdesk products solve this problem, but they are priced per agent per month and assume a dedicated support team, which places them beyond the reach of a three-person business.

ServiceDesk AI is a cloud-native Software-as-a-Service application that closes this gap. Every complaint becomes a ticket carrying a reference number, an owner, a category, a priority and a tracked status — visible to the customer, actionable by the support agent and measurable by the administrator. Customers can reach the desk from a web portal or, primarily, by sending an ordinary WhatsApp message: the message becomes a ticket, is classified automatically, and receives an immediate acknowledgement; every agent reply is delivered back to the same chat. The system implements three roles with role-based access control enforced in the API, a guarded six-state ticket lifecycle, threaded conversations with staff-only notes, full-text search, persistent notifications, role-specific dashboards, exportable reports, an audit log, and an administration module.

An AI layer assists the agent without replacing them: incoming tickets are classified by category, priority and sentiment; an agent can request a summary, suggested resolution steps, a draft reply, or a knowledge-base answer with citations retrieved by vector similarity from PostgreSQL. The AI is deliberately kept off the critical path — classification runs after the ticket is committed, every provider failure converges on a rule-based fallback, and nothing the AI produces reaches a customer without an agent sending it.

The application is deployed and publicly reachable: the React frontend on Vercel, the Express/TypeScript API and a managed PostgreSQL 16 instance on Render (provisioned from a version-controlled Blueprint), and the WhatsApp channel through Meta's WhatsApp Cloud API. Correctness is evidenced by 161 automated tests executed against a real PostgreSQL database using the production migrations, all passing, and by a continuous-integration pipeline that runs them on every push. The complete workflow was additionally exercised in a browser across all three roles and from a phone over WhatsApp.

Keywords: Cloud Computing, Software as a Service, Platform as a Service, Customer Support, Ticket Management, Role-Based Access Control, REST API, PostgreSQL, pgvector, React, Node.js, WhatsApp Cloud API, Retrieval-Augmented Generation, Graceful Degradation.

3. Table of Contents

Generated from the document headings. In Microsoft Word, right-click the table and choose “Update Field” (or press F9) to refresh page numbers.

Contents at a glance

Section	Title	Section	Title
Part A	Deliverables map (R2, R3, checklist, accounts)	13	Database Design and ER diagram
2	Abstract	14	Module Description
4	Introduction	15	User Interface Design (live screenshots)
5	Problem Statement	16	Implementation
6	Existing System	17	Testing (161 automated tests)
7	Proposed System	18	Deployment (Render, Vercel, Meta)
8	Requirement Analysis	19	Results and Discussion
9	Functional Requirements and lifecycle	20	Future Enhancements
10	Non-Functional Requirements	21	References
11	System Architecture	22	Appendices A-E: demo script, API catalogue, test index, environment, repository map
12	Cloud Architecture		

4. Introduction

4.1 Background

Cloud computing has changed how small organisations obtain software. Capabilities that once required a server room, a database administrator and a capital purchase are now consumed over the internet, billed by usage, and maintained by the provider. The service models are conventionally described in three layers: Infrastructure as a Service, where virtual machines and storage are rented; Platform as a Service, where an application is deployed onto a managed runtime; and Software as a Service, where finished software is used directly through a browser.

This project sits at all three layers at once. The delivered product is Software as a Service: a small business signs in through a browser and uses a support desk without installing anything. It is built on Platform as a Service: the frontend and the API are deployed to managed platforms that own the runtime, scaling and transport security. It stores its data in a Database as a Service instance, so no member of the team administers a database server. Being able to explain and defend each of these choices is a substantial part of the value of the exercise.

4.2 Motivation

Customer support is a problem every business has, but the tooling available is sharply divided. Large organisations run dedicated helpdesk platforms; very small ones fall back on a shared mailbox and a spreadsheet. The middle ground is poorly served because the commercial products are priced on the assumption that the buyer employs support staff full time. This is a genuine and unglamorous problem, and it exercises every capability the course requires — multiple roles, CRUD over a non-trivial entity, search, reporting, notifications and administration — because the domain demands them rather than having them bolted on.

A second motivation shaped the final design. In India, a small business's customers do not open a support portal; they send a WhatsApp message. The system was therefore built so that WhatsApp is the primary intake channel and the web application is where staff work.

4.3 Objectives

#	Objective (as specified for Project 19)	How it is satisfied
1	Develop a scalable cloud-native SaaS application	Three independently deployable tiers; a stateless API that permits horizontal replication; CDN-served frontend
2	Implement secure role-based authentication	JWT sessions, bcrypt hashing, roles enforced server-side in two layers, 39 tests on authentication and authorisation
3	Provide dashboards and analytical reports	Three role-specific dashboards and seven reports, computed from live data, with CSV export
4	Deploy the application on a public cloud platform	Vercel and Render, reachable at https://servicedesk-ai-two.vercel.app ; infrastructure defined in render.yaml
5	Demonstrate collaborative software engineering practices	One repository with a documented history, pull requests with automated review, versioned migrations, CI on every push, 161 automated tests

Table 4.1 — Project objectives and the means of satisfying them

4.4 Organisation of this document

Part A maps every official deliverable to its evidence. Sections 5 to 10 establish what the system must do and why. Sections 11 to 15 describe how it is structured. Sections 16 to 18 describe how it was built, verified and deployed. Section 19 discusses the outcome candidly, including defects found during development and remaining limitations. The appendices contain the live demonstration script, the API catalogue, the test index and the deployment configuration.

5. Problem Statement

The official specification states the problem in a single sentence: *“Manage customer complaints and automate ticket handling.”* A single sentence is not a specification, so requirement analysis was performed to establish what it means in the operating reality of a small business. Four concrete failures were identified, each mapping to a feature of the system.

Observed failure	What actually happens	Addressed by
No single intake point	Requests arrive by WhatsApp, email and telephone and are never consolidated	One ticket store with a generated reference; WhatsApp messages become tickets automatically
No ownership	A complaint is answered twice by different people, or not at all	Assignment, an agent queue, notification on assignment
No status or history	The customer cannot see progress and follows up repeatedly	A six-state lifecycle, a threaded conversation, a per-ticket timeline, STATUS on WhatsApp
No measurement	The business cannot say how many issues arrived or how long they took	Dashboards and reports computed from live data

Table 5.1 — Decomposition of the problem statement

On the word “automate”. It admits two readings. Procedural automation — creating the record, assigning a reference, timestamping events, advancing status when an agent replies, notifying, acknowledging a WhatsApp message — is implemented fully and depends on no AI. Intelligent automation — interpreting the content of a complaint — is also implemented, as classification, summarisation, drafting and knowledge-base answers, but with a deliberate boundary: the system automates the preparation of a response, never the decision to send one.

6. Existing System

Three approaches are in common use. Each was examined for what it provides and where it fails.

6.1 Shared inbox or personal WhatsApp

- No ticket identity: a customer calling about an earlier message has nothing to quote.
- No ownership: two staff members reply independently to the same message.
- The only state is read or unread; “waiting for the customer” cannot be expressed.
- History is scattered across threads and phones; no reporting is possible.

6.2 Spreadsheet tracker

- Updated by hand, so it drifts out of date immediately and is trusted less over time.
- Offers the customer nothing; no access control; no record of who changed what.
- Unusable beyond a few dozen rows, which is exactly when the business needs it.

6.3 Commercial helpdesk platforms

Zendesk, Freshdesk and Jira Service Management solve the problem thoroughly for organisations of sufficient size. They are priced per agent per month, assume a dedicated support function, require substantial configuration before delivering value, and provide far more than a three-person team needs.

6.4 Comparison

Capability	Shared inbox	Spreadsheet	Commercial helpdesk	ServiceDesk AI
Ticket reference	No	Manual	Yes	Yes
Assigned owner	No	Manual	Yes	Yes
Customer-visible status	No	No	Yes	Yes — web and WhatsApp STATUS
Access control by role	No	No	Yes	Yes
Search and filtering	Limited	Limited	Yes	Yes, full-text
Reporting	No	Manual	Yes	Yes
Audit trail	No	No	Yes	Yes
Automatic classification	No	No	Partly	Yes, with fallback
WhatsApp intake	Informal	No	Paid add-on	Built in
Cost at three agents	Free	Free	Recurring per agent	Free-tier hosting
Setup effort	None	Low	High	Low

Table 6.1 — Existing approaches compared with the proposed system

The gap: a small business needs the structure of a helpdesk without its pricing or complexity — ticket identity, ownership, visible status, access control and measurement, delivered with the setup effort of a spreadsheet, on the channel its customers already use.

7. Proposed System

7.1 Overview

ServiceDesk AI is a multi-user, cloud-hosted SaaS application. Every complaint becomes a ticket with an owner, a category, a priority and a tracked status. The customer can see it (on the web or by WhatsApp), the agent can act on it, and the administrator can measure it.

7.2 Workflow

1. A customer sends a WhatsApp message (or raises a ticket on the web). The ticket is stored immediately with status NEW and a reference such as SD-1007; the customer receives an acknowledgement naming that reference within about a second.
2. Classification runs in the background after the database commit: category, priority, sentiment and a one-line summary. A failure here cannot affect the ticket.
3. The ticket enters the agent queue ordered by priority, filterable and searchable across every customer.
4. An agent takes ownership (OPEN), and may ask the assistant for a summary, resolution steps, a knowledge-base answer with citations, or a draft reply. The agent edits and sends. Sending records first-response time, moves the ticket to IN_PROGRESS, and delivers the reply to the customer's WhatsApp as plain text.
5. The customer is notified and can respond in the thread or the chat. A reply on a ticket that was WAITING_FOR_CUSTOMER returns it to IN_PROGRESS.
6. When fixed, the agent marks the ticket RESOLVED. The customer closes it (on the web or with CLOSE on WhatsApp), or reopens it if the problem persists.
7. Throughout, every change writes a timeline event and every privileged action writes an audit record, which feed the dashboards and reports.

7.3 What makes the system cloud-native

Property	Realisation
Managed data tier	PostgreSQL is consumed as a service on Render with its own backups and availability; nobody on the team administers a database server.
Stateless application tier	The API holds no session state — sessions are signed tokens and all other state lives in the database — so the platform can run several replicas without sticky sessions.
Edge-distributed presentation tier	The frontend is a static build served from Vercel's CDN close to the user, scaling independently of the API.
Infrastructure as code	render.yaml provisions the database and API together, generates secrets, and wires the connection string; the same file recreated the whole stack in a different region in minutes.
Continuous delivery	A push to main runs the CI pipeline and redeploys both tiers automatically.
Public reachability	Addressable from any browser and any WhatsApp phone; no dependence on localhost.

Table 7.1 — Cloud-native properties

7.4 Scope

In scope

- Registration, login, profile management for three roles; role-based access enforced in the API
- Full ticket lifecycle: create, read, update, delete, assign, guarded status transitions
- WhatsApp as a two-way channel: intake, keyword commands (HELP, STATUS, NEW, CLOSE), agent reply delivery
- Threaded conversation with internal notes; search, filtering, pagination; notifications
- Role-specific dashboards, seven reports, CSV export; administration of users, roles, categories, audit log
- AI classification, summarisation, resolution steps, agent-reviewed drafts, and a knowledge base with retrieval-augmented answers
- Responsive interface; public cloud deployment; containerised local stack; CI

Out of scope (deliberate)

- Payment or subscription billing; telephony; native mobile applications; multi-language support
- File attachments and cloud object storage (the schema anticipates them; see Section 19.5)
- Fully autonomous AI replies sent without agent approval — a design position, defended in Section 16.6

8. Requirement Analysis

8.1 Sources

Ref	Document	Requirements contributed
A1	BCSE408L Cloud-Native SaaS Project Guidelines	Nine mandatory technical requirements; team, repository, deployment, AI-usage and integrity rules; review scheme; submission checklist
A2	BCSE408L Project Review Schedule and Report Template	Deliverables for each review; the 22-section report structure; the final demonstration checklist
A3	BCSE408L Cloud SaaS Project Requirements, Project 19	Overview, problem statement, five objectives, three users, ten functional and six non-functional requirements, five modules, cloud requirements, eight deliverables

Table 8.1 — Source documents

8.2 Stakeholders and actors

Actor	Responsibilities and interest
Customer	Raises tickets (web or WhatsApp), follows progress, replies in the thread. Sees only their own tickets.
Support Agent	Works a prioritised queue across all customers; updates status, priority, category, assignment; adds internal notes; uses the assistant; replies reach WhatsApp customers automatically.
Admin	Every agent capability plus users, roles, categories, global reports, the audit log and the WhatsApp console.
Course evaluator	Requires demonstrable satisfaction of every mandatory requirement and that both members can explain or modify the implementation.

Table 8.2 — Actors

8.3 Discrepancies identified between the sources

- **AI is not a stated requirement.** Only the project title contains the term. Every AI capability is therefore an addition, and the design rule adopted is that no mandatory requirement may depend on it — hence classification after commit and a working offline fallback.
- **“Cloud storage where applicable” (A3).** Attachments were scoped out; this clause is recorded as not satisfied in Section 19.5 rather than presented as complete.
- **A3 omits “Working Demonstration”,** which A1 and A2 require. The union of all three documents was treated as binding.
- **A1 requires that any member can explain or modify any line.** This constrained how the code was written: no unnecessary abstraction, module names mirroring the specification, developer documentation alongside the code.

8.4 Constraints

ID	Constraint	Design consequence
C-1	Publicly accessible; localhost-only not acceptable	Three-tier deployment on managed platforms; no component assumes a local address
C-2	Permitted hosting platforms only	Vercel and Render, both on the permitted list
C-3	Both members must explain and modify the code	Plain structure; one place per rule (e.g. the lifecycle table); extensive comments
C-4	Original work	Design system and every component written for this project; no UI kit or template
C-5	Free tiers suspend idle services	Health endpoint for scheduled pinging; visible loading states
C-6	No secret in the repository or the browser bundle	All secrets as environment variables; AI and WhatsApp keys used only server-side; Vercel holds exactly one public variable
C-7	WhatsApp renders no Markdown	Every outbound message passes through a plain-text formatter

Table 8.3 — Constraints

8.5 Feasibility

Technically feasible: the stack is mature and the whole system was developed and tested against a real PostgreSQL before deployment. Economically feasible: every platform used — Vercel, Render, Meta’s WhatsApp Cloud API, GitHub Actions — was used on its free tier, and the total cost of the deployed system is zero. Operationally feasible: a support desk is a familiar workflow and WhatsApp needs no training at all. Schedule: delivered.

9. Functional Requirements

The ten headings are the functional requirements as stated for Project 19, elaborated into testable behaviour. The test identifiers refer to Section 17 and Appendix C.

ID	Requirement	Implemented behaviour	Tests
FR-1	User registration and login	Register (name, unique case-insensitive email, password ≥ 8 chars with letter and digit); bcrypt hash only; sign in; refresh via httpOnly cookie; sign out; self-registration always creates a Customer	A-01 – A-13
FR-2	Profile management	View and edit own name and phone; change password with current password; cannot change own role or email	Browser-verified
FR-3	Role-based access control	Three roles; every protected route checks role before the handler; ticket handlers also check ownership; 404 rather than 403 for another customer's ticket; deactivated accounts refused immediately	R-01 – R-26
FR-4	CRUD operations	Tickets, messages, users, categories, knowledge articles; delete cascades to messages and events; status changes validated against the lifecycle table	T-01 – T-33
FR-5	Search and filtering	Filter by status, priority, category, agent, sentiment, channel, date; full-text search on subject, description and reference via a GIN index; paginated, sortable; customer results always scoped to own tickets	T-27 – T-33
FR-6	Dashboard	Customer, Agent and Admin dashboards; every figure aggregated at request time — no hard-coded value	Browser-verified, live
FR-7	Notifications	Persistent notifications on assignment, reply and status change; unread badge; mark one or all read; WhatsApp customers also receive status notifications on their phone	T-13, WA-08
FR-8	Report generation	By status, priority, category; sentiment; agent workload; created-vs-resolved trend; resolution time by category; RFC 4180 CSV export with formula-injection protection	Browser-verified
FR-9	Cloud database integration	Managed PostgreSQL 16 on Render; versioned SQL migrations applied on every deploy; the frontend never touches the database	Whole suite; live health check
FR-10	Administrator module	List, search, create users; change role; activate/deactivate; cannot remove own admin role; manage categories; audit log of every privileged action; WhatsApp console	R-23 – R-26

Table 9.1 — Functional requirements and their evidence

9.11 Ticket lifecycle

The lifecycle is the central business rule. Permitted transitions are defined once, in a single table in backend/src/modules/tickets/lifecycle.ts, and validated on every status change; the database additionally constrains resolved_at to be set only in RESOLVED or CLOSED.

Current state	Permitted next states	Notes
NEW	OPEN, IN_PROGRESS, CLOSED	Newly submitted, not yet triaged
OPEN	IN_PROGRESS, WAITING_FOR_CUSTOMER, RESOLVED, CLOSED	Triaged and assigned
IN_PROGRESS	WAITING_FOR_CUSTOMER, RESOLVED, CLOSED	An agent is working on it
WAITING_FOR_CUSTOMER	IN_PROGRESS, RESOLVED, CLOSED	Returns to IN_PROGRESS automatically when the customer replies
RESOLVED	CLOSED, OPEN	Customer confirms (CLOSE) or reopens
CLOSED	— (terminal)	Reopening creates history rather than mutating the past

Table 9.2 — Ticket lifecycle transitions

9.12 WhatsApp conversation model

Customer sends	System does
Any description of a problem	Creates the customer account if the number is new, raises a ticket, classifies it, replies with the reference
Another message while a ticket is open	Appends it to that ticket and confirms; WAITING_FOR_CUSTOMER returns to IN_PROGRESS
STATUS	Lists their open tickets and states
HELP	Shows the commands
NEW <description>	Raises a separate ticket even when one is open
CLOSE	Closes a RESOLVED ticket, or explains why it cannot
(agent replies or changes status in the web app)	Delivered to the chat as plain text, prefixed with the reference

Table 9.3 — WhatsApp keywords

10. Non-Functional Requirements

ID	Requirement	Concrete mechanism
NFR-1	Security	bcrypt(12); JWT with short-lived access token in memory and httpOnly refresh cookie scoped to /api/auth; two-layer RBAC; Zod validation on every input; parameterised SQL only; Helmet headers; CORS allow-list; rate limiting; webhook signature verification (HMAC-SHA256 for Meta, HMAC-SHA1 for Twilio) with constant-time comparison; identical error for unknown email and wrong password; secrets redacted from logs
NFR-2	Scalability	Stateless API; bounded connection pool; server-side pagination on every list; CDN frontend; AI and WhatsApp delivery off the request path
NFR-3	Availability	Managed services; /api/health checks process and database; graceful SIGTERM shutdown; AI degrades to rules rather than failing; WhatsApp delivery failure never rolls back the ticket change; migrations run before the server binds
NFR-4	Performance	Index on every filter column; GIN full-text index; HNSW vector index; pagination; client-side caching with TanStack Query; classification after commit so ticket creation returns immediately
NFR-5	Reliability	Database CHECK and FK constraints; single lifecycle table; transactions for status change + assignment; idempotent webhooks via UNIQUE provider message id; audit log; 161 automated tests against a real database
NFR-6	Usability	One hand-built design system; responsive from 375 px; explicit loading, empty and error states; plain-language error messages; WhatsApp replies in plain text with no jargon

Table 10.1 — Non-functional requirements

11. System Architecture

The system is a three-tier web application with two external integrations. The browser and the customer's WhatsApp are the clients; the API is the only component that touches the database; and every external dependency sits behind an adapter with an offline implementation.

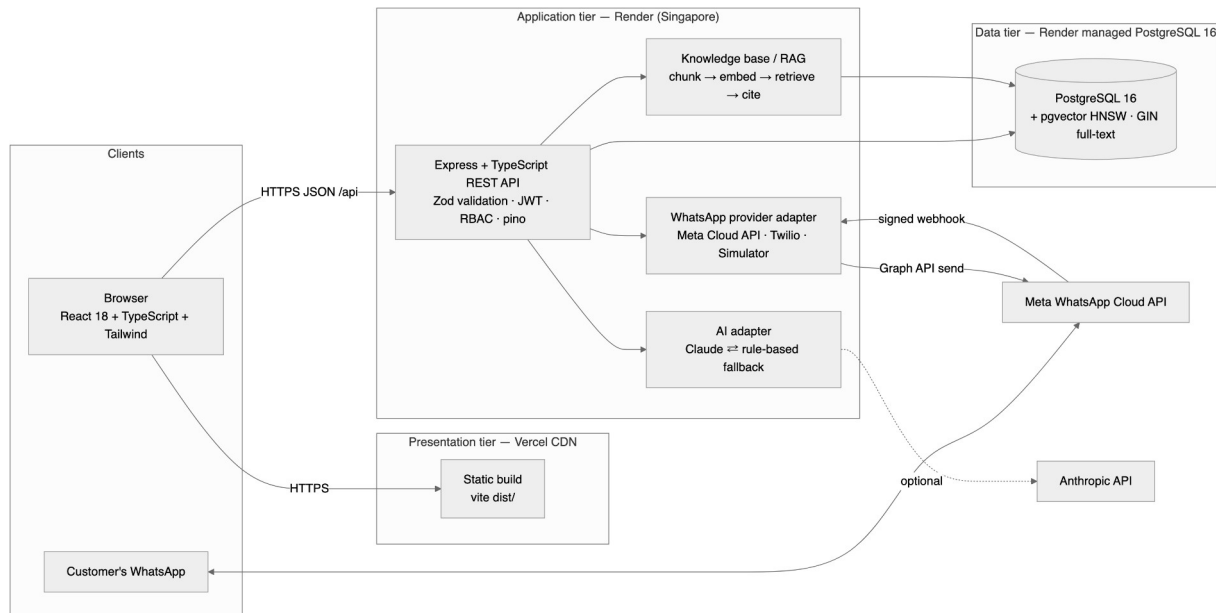


Figure 11.1 — System architecture

11.1 Tiers

Tier	Technology	Responsibility
Presentation	React 18, TypeScript, Vite, Tailwind CSS, TanStack Query, React Router, Recharts	Role-aware single-page application. Holds the access token in memory only; calls the API over HTTPS; never sees a secret or the database.
Application	Node.js 20, Express, TypeScript, Zod, jsonwebtoken, bcryptjs, pino, Helmet, express-rate-limit	REST API. Validates every input, authenticates, authorises, enforces business rules, writes audit and timeline records, orchestrates AI and WhatsApp adapters.
Data	PostgreSQL 16 with pgvector; versioned SQL migrations	Single source of truth. Constraints enforce invariants independently of application code; GIN index for full-text search; HNSW index for vector similarity.
Integrations	Meta WhatsApp Cloud API (Twilio and a simulator as alternatives); Anthropic Claude (rule-based fallback); OpenAI-compatible embeddings (deterministic fallback)	Each behind a small interface. The application works with none of them configured.

Table 11.1 — Tiers and responsibilities

11.2 Request path

1. The request passes the HTTP logger (request id assigned), Helmet, the CORS allow-list, body parsing with raw-body capture (for webhook signatures), and the rate limiter.

2. `requireAuth` verifies the JWT and re-reads the user row, so a deactivated account is refused on its very next request.
3. `requireRole` checks the route's permitted roles; the handler then checks ownership of the specific record where relevant.
4. `validate(schema)` parses body, query and params with Zod; invalid input returns 400 with field-level errors.
5. The handler runs parameterised SQL through a shared pool; multi-step writes use `withTransaction`.
6. Side effects that must not block the response — AI classification, WhatsApp delivery, notifications — run after the commit and record their own outcome.
7. Every response uses one envelope: `{ success, data, meta }` or `{ success: false, error: { code, message, details } }`.

11.3 Backend module structure

Directory	Contents
modules/authentication	Register, login, refresh, logout, current user
modules/user-management	Profile, password change, admin user CRUD, role and status changes
modules/tickets	Ticket CRUD, lifecycle.ts, assignment, messages, events, search filters, classifyAndStore
modules/reports	Dashboard aggregates for three roles; seven reports; CSV export
modules/administration	Categories; audit-log listing
modules/notifications	Per-user notification listing and read state
modules/ai	Draft reply, summary, resolution steps, grounded answer, suggestion history, AI health
modules/whatsapp	Webhook (POST + verification GET), inbound conversation logic, admin console endpoints
modules/knowledge	Knowledge-base article CRUD, indexing, vector search, embedding status
services/ai, services/embeddings, services/whatsapp	Provider adapters and their fallbacks; plain-text formatter
middleware	auth, rbac, validate, errorHandler
db	pool, migrate, seed, migrations/001-005
observability	pino logger with redaction; HTTP request logger
openapi	OpenAPI 3 specification generated from the Zod schemas; served at <code>/api/openapi.json</code> and <code>/api/docs</code>

Table 11.2 — Backend structure

12. Cloud Architecture

The deployed topology is shown below exactly as it runs today. Every box is a managed service; the team operates no server.

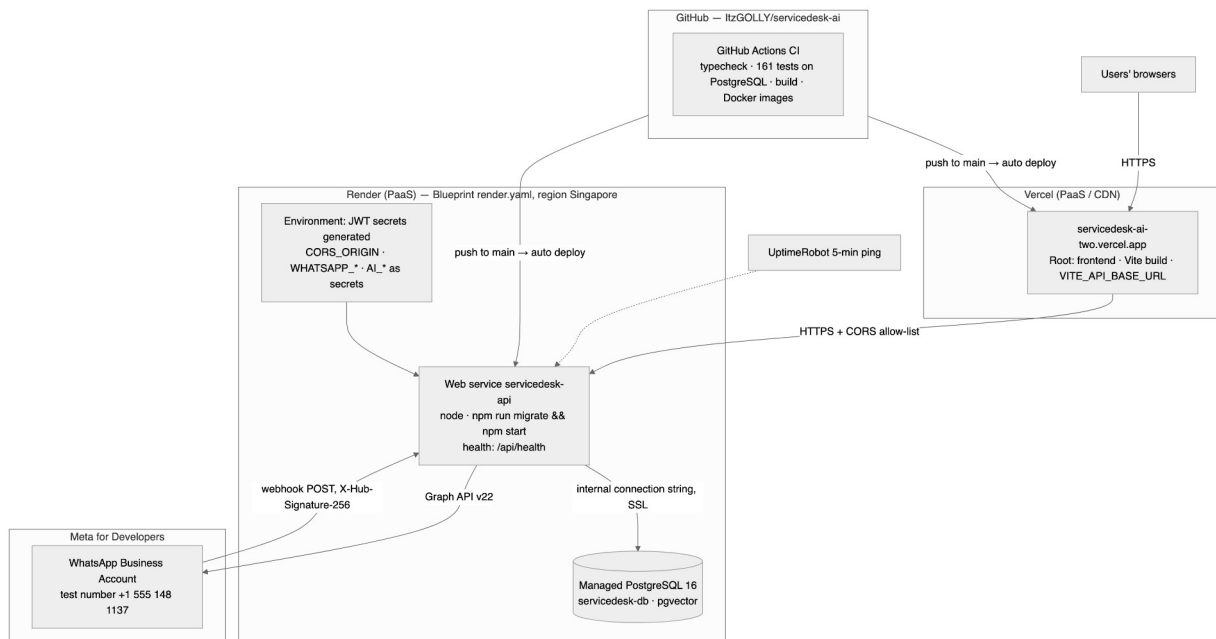


Figure 12.1 — Cloud deployment architecture

12.1 Service model mapping

Layer	Service	Cloud model	Why this choice
Frontend	Vercel	PaaS / CDN	Static Vite build cached at the edge; automatic deploy on push; free tier; on the permitted list
API	Render Web Service (Singapore)	PaaS	Node runtime managed by the platform; Blueprint support; health-checked; free tier; nearest region to India on the permitted list
Database	Render PostgreSQL 16	DBaaS	Managed backups, SSL, the pgvector extension pre-installed; same region and private network as the API
Messaging	Meta WhatsApp Cloud API	SaaS API	Official WhatsApp Business Platform; free test number sends real free-form replies within the customer-initiated 24-hour window
CI	GitHub Actions	SaaS	Typecheck, tests on a real PostgreSQL service container, frontend build, Docker image builds on every push and pull request
Uptime	UptimeRobot	SaaS	Five-minute ping of <code>/api/health</code> so the free instance does not sleep during the demonstration

Table 12.1 — Cloud services

12.2 Infrastructure as code

render.yaml at the repository root defines the database and the web service together: plan, region, Node build (npm ci --include=dev && npm run build), a start command that applies migrations before the server binds (npm run migrate && npm start), the health-check path, generated JWT secrets, the database connection string wired automatically, and every third-party secret marked sync: false so it is entered once in the dashboard and never stored in the repository. During deployment the first attempt failed because the database defaulted to a different region from the API; adding one line (region: singapore) and re-applying the Blueprint recreated the stack correctly — which is precisely the value of defining infrastructure in a file.

12.3 Security boundaries

- The database accepts connections only over Render's private network (the API uses the internal connection string over SSL).
- The browser holds exactly one configuration value, VITE_API_BASE_URL. No AI key, WhatsApp token or database credential ever reaches the client.
- CORS is an exact-origin allow-list, required because the refresh cookie is sent with credentials.
- Meta signs every webhook delivery with HMAC-SHA256 over the raw body using the app secret; unsigned or mis-signed requests are rejected with 403 before any processing.
- Render redeploys on every push to main, so the running system never diverges from the reviewed code.

12.4 Scaling path

Because the API is stateless, Render can run additional instances behind its load balancer with no code change; the bounded per-instance pool (max 10) keeps total database connections predictable. The frontend scales with the CDN. The database is the only vertical component; at small-business volumes (thousands of tickets) a single managed instance is more than sufficient, and read replicas are a configuration step when needed.

13. Database Design

13.1 Why a relational database

A support desk is a set of entities with strict relationships and invariants: a message belongs to exactly one ticket; a ticket has exactly one customer; a status can only change along defined paths; a resolved timestamp only makes sense in a resolved state. PostgreSQL enforces all of this in the schema, so the invariants hold even if application code has a defect. Full-text search and — through pgvector — vector similarity search are available in the same engine, which avoided introducing a separate search or vector database.

13.2 Entity relationship diagram

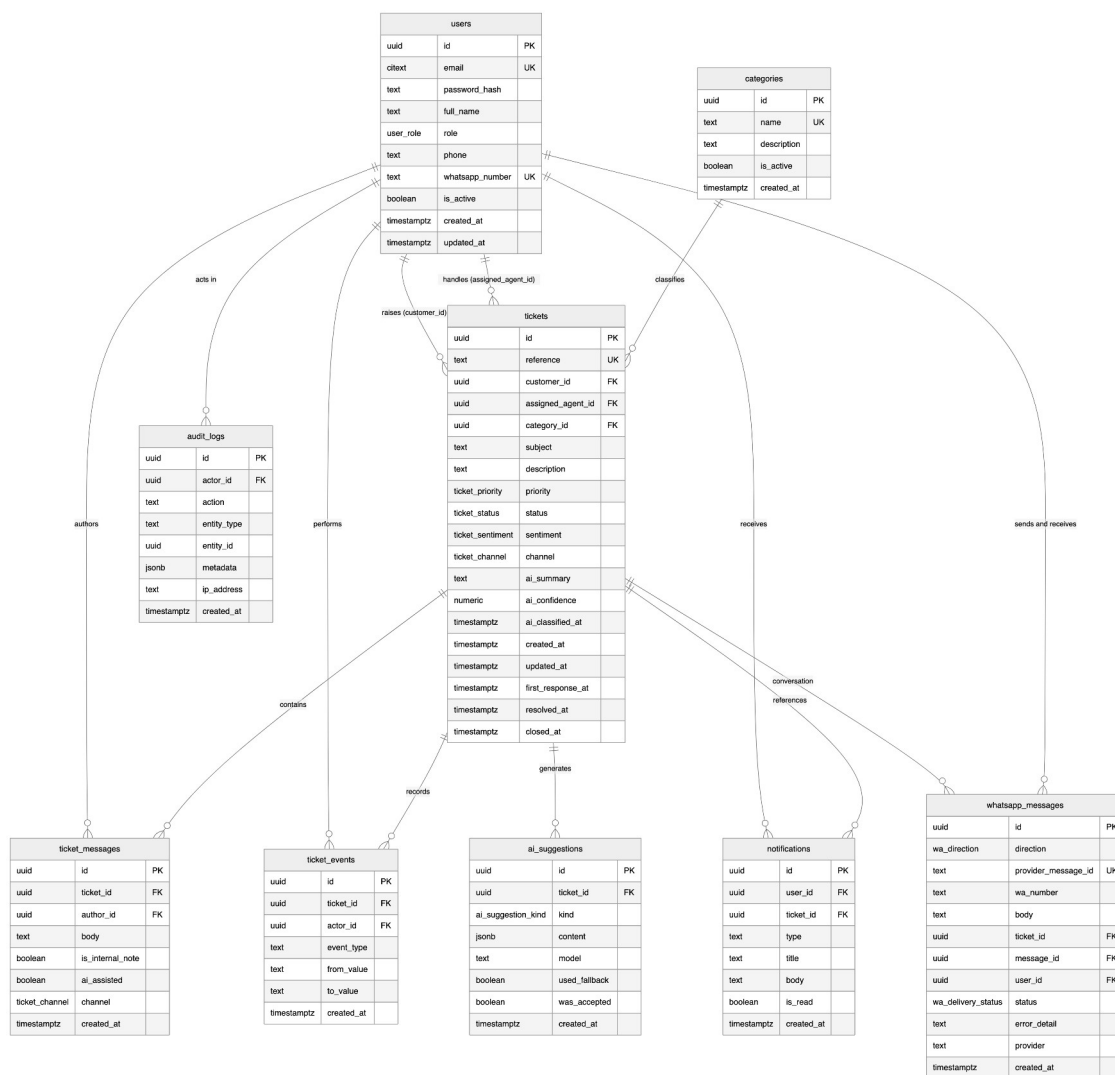


Figure 13.1 — ER diagram (derived from migrations 001–003)

13.3 Tables

Table	Purpose	Notable design decisions
users	Every account, all roles	CITEXT unique email; whatsapp_number unique and nullable; is_active soft-deactivation preserves history
categories	Ticket taxonomy	Inactive categories hidden from new tickets but retained on old

Table	Purpose	Notable design decisions
		ones; delete refused when in use (409)
tickets	The core entity	reference SD-nnnn from a sequence; customer FK RESTRICT, agent and category FK SET NULL; channel WEB/WHATSAPP; ai_confidence CHECK 0–1; resolved_at CHECK against status
ticket_messages	Conversation	is_internal_note filtered out for customers; ai_assisted records that a draft was used; channel of origin
ticket_events	Timeline	Append-only; every status/assignment/priority change with from/to values
ai_suggestions	Every AI output	kind enum (CLASSIFICATION, DRAFT_REPLY, SUMMARY, RESOLUTION_STEPS, GROUNDED_ANSWER); used_fallback and was_accepted make AI usage measurable and honest
notifications	Per-user inbox	Scoped by user_id; 404 for another user's notification
audit_logs	Privileged actions	actor, action, entity, JSONB metadata, IP; append-only
whatsapp_messages	Channel log	UNIQUE provider_message_id inserted before any work — a replayed webhook cannot create a second ticket; status and error_detail make failures visible in the console
kb_articles / kb_chunks / kb_retrievals	Knowledge base	Articles chunked (~700 chars, 100 overlap); VECTOR(384) embeddings; HNSW cosine index; every retrieval recorded for evaluation

Table 13.1 — Tables

13.4 Enumerated types

Type	Values
user_role	CUSTOMER, AGENT, ADMIN
ticket_status	NEW, OPEN, IN_PROGRESS, WAITING_FOR_CUSTOMER, RESOLVED, CLOSED
ticket_priority	LOW, MEDIUM, HIGH, URGENT
ticket_sentiment	POSITIVE, NEUTRAL, NEGATIVE
ticket_channel	WEB, WHATSAPP
ai_suggestion_kind	CLASSIFICATION, DRAFT_REPLY, SUMMARY, RESOLUTION_STEPS, GROUNDED_ANSWER
wa_direction / wa_delivery_status	INBOUND, OUTBOUND / PENDING, SENT, DELIVERED, READ, FAILED

Table 13.2 — Enumerated types

13.5 Indexes and migrations

- B-tree indexes on every column used as a filter or join: status, priority, category_id, assigned_agent_id, customer_id, created_at, user_id on notifications, ticket_id on messages and events.

- A GIN index over `to_tsvector(subject || description || reference)` for full-text search.
- An HNSW index (`m = 16`, `ef_construction = 64`) on `kb_chunks.embedding` for cosine similarity. IVFFlat was tried first and replaced: on a near-empty table it silently returned one row where four matched, which a pinned test now guards against.
- Five migrations (001 core schema, 002 WhatsApp, 003 knowledge base, 004 HNSW, 005 grounded-answer kind) applied in order by a 60-line runner that records each file in `schema_migrations`. The same files run in CI, in Docker, and on Render.

14. Module Description

The five modules suggested in the specification are implemented under those names; three further modules were added to realise the WhatsApp channel and the assistant.

Module	Key capabilities	Main files
Authentication	Register, login, refresh, logout, current user; short-lived access token, httpOnly refresh cookie; instant deactivation	authentication/auth.routes.ts, middleware/auth.ts
User Management	Own profile and password; admin list/search/create; role change; activate/deactivate; self-lockout prevention	user-management/users.routes.ts
Tickets (core)	CRUD, lifecycle, assignment, messages and internal notes, timeline, search and filters, background classification	tickets/tickets.routes.ts, tickets.service.ts, lifecycle.ts
Reports & Dashboard	Three dashboards; seven reports; CSV export; AI-usage statistics	reports/dashboard.routes.ts, reports.routes.ts
Administration	Categories; audit log; WhatsApp console; AI health	administration/*, whatsapp.routes.ts (admin)
WhatsApp channel	Signed webhook, idempotency, conversation model, keyword commands, plain-text formatter, provider adapters (Meta, Twilio, simulator), delivery of agent replies and status changes	whatsapp/*, services/whatsapp/*
AI assistance	Classification after commit; draft reply, summary, resolution steps; provider adapter with rule-based fallback; every output persisted with used_fallback	ai/ai.routes.ts, services/ai/*
Knowledge base (RAG)	Article CRUD, chunking, embeddings, pgvector retrieval, lexical fallback, grounded answers with verified citations, retrieval logging	knowledge/*, services/embeddings/*

Table 14.1 — Modules

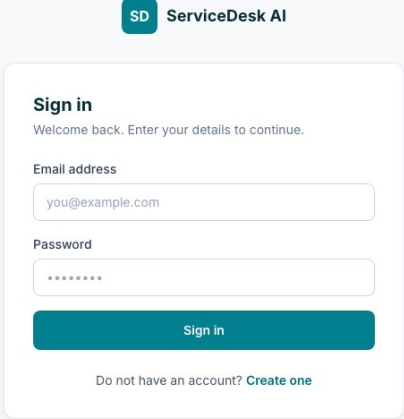
14.1 Frontend structure

Area	Pages / components
Public	Landing, Login, Register
Customer	Dashboard, My tickets, Create ticket, Ticket detail (thread), Notifications, Profile
Agent	Agent dashboard, Queue (filters, search, pagination), Ticket detail with AI panel (draft, summary, steps, knowledge answer), assignment and status controls
Admin	Admin dashboard (charts), All tickets, Users, Categories, WhatsApp console, Reports (with CSV), Activity log
Shared	AppShell (role-aware navigation, unread badge), design system in index.css, lib/api.ts (single API client, token handling, base-URL normalisation)

Table 14.2 — Frontend structure

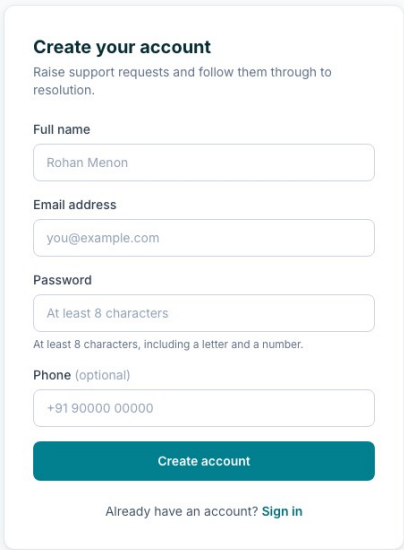
15. User Interface Design

All screenshots below were captured from the live deployment on 19 September 2026 with real data. The design system is hand-built on Tailwind: one teal accent, a neutral palette, consistent cards, and explicit loading, empty and error states on every data view.



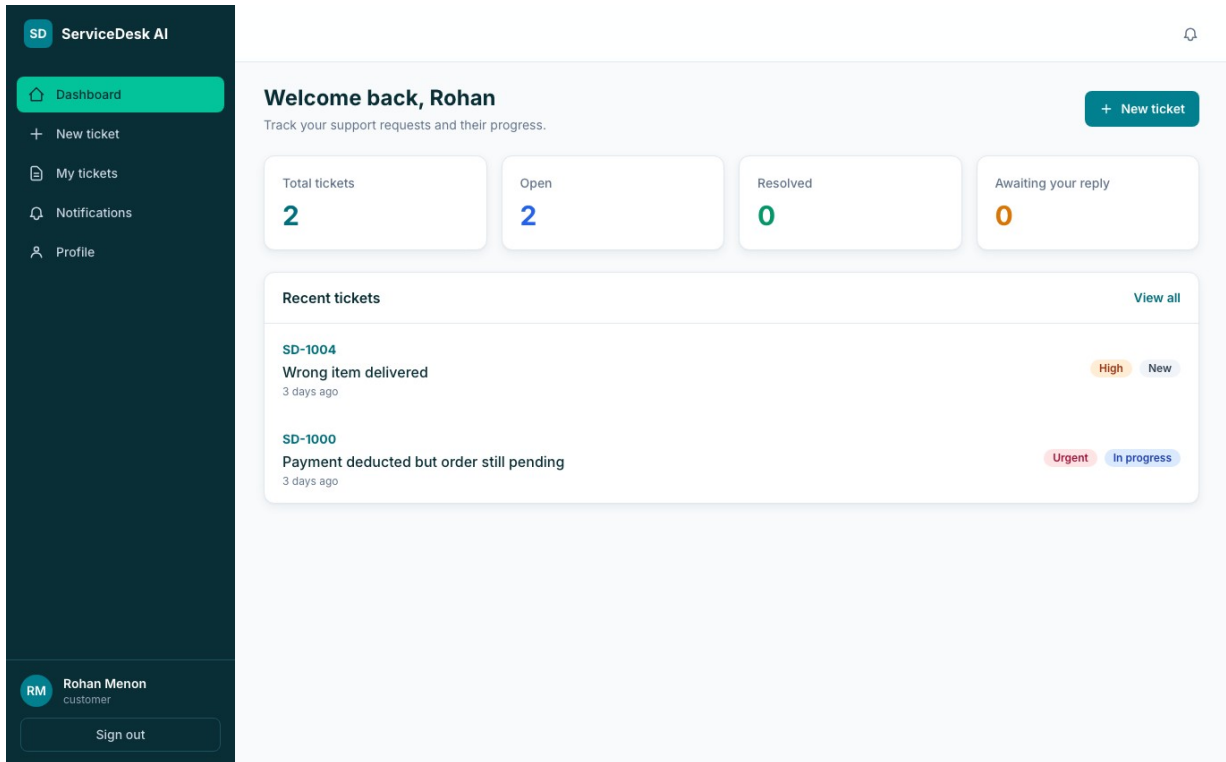
The sign-in form is centered on a light gray background. At the top, there is a teal square logo with the letters 'SD' in white, followed by the text 'ServiceDesk AI'. Below this is a white card with rounded corners. Inside the card, the heading 'Sign in' is in bold. Underneath is a subtext: 'Welcome back. Enter your details to continue.' The form contains two input fields: 'Email address' with the placeholder 'you@example.com' and 'Password' with a masked input '*****'. A teal 'Sign in' button is positioned below the password field. At the bottom of the card, there is a link: 'Do not have an account? [Create one](#)'.

Figure 15.1 — Sign in



The registration form is centered on a light gray background. At the top, there is a teal square logo with the letters 'SD' in white, followed by the text 'ServiceDesk AI'. Below this is a white card with rounded corners. Inside the card, the heading 'Create your account' is in bold. Underneath is a subtext: 'Raise support requests and follow them through to resolution.' The form contains four input fields: 'Full name' with the placeholder 'Rohan Menon', 'Email address' with the placeholder 'you@example.com', 'Password' with the placeholder 'At least 8 characters' and a subtext 'At least 8 characters, including a letter and a number.', and 'Phone (optional)' with the placeholder '+91 90000 00000'. A teal 'Create account' button is positioned below the phone field. At the bottom of the card, there is a link: 'Already have an account? [Sign in](#)'.

Figure 15.2 — Registration (always creates a Customer)



The screenshot shows the ServiceDesk AI Customer Dashboard. On the left is a dark sidebar with the 'ServiceDesk AI' logo and navigation links: Dashboard (highlighted), New ticket, My tickets, Notifications, and Profile. The main content area has a header 'Welcome back, Rohan' with a 'New ticket' button. Below this are four summary cards: 'Total tickets' (2), 'Open' (2), 'Resolved' (0), and 'Awaiting your reply' (0). A 'Recent tickets' section follows, listing two tickets: 'SD-1004 Wrong item delivered' (High priority, New status) and 'SD-1000 Payment deducted but order still pending' (Urgent priority, In progress status). At the bottom, the user profile for Rohan Menon is shown with a 'Sign out' button.

ServiceDesk AI

Dashboard

- New ticket
- My tickets
- Notifications
- Profile

Welcome back, Rohan

Track your support requests and their progress.

[+ New ticket](#)

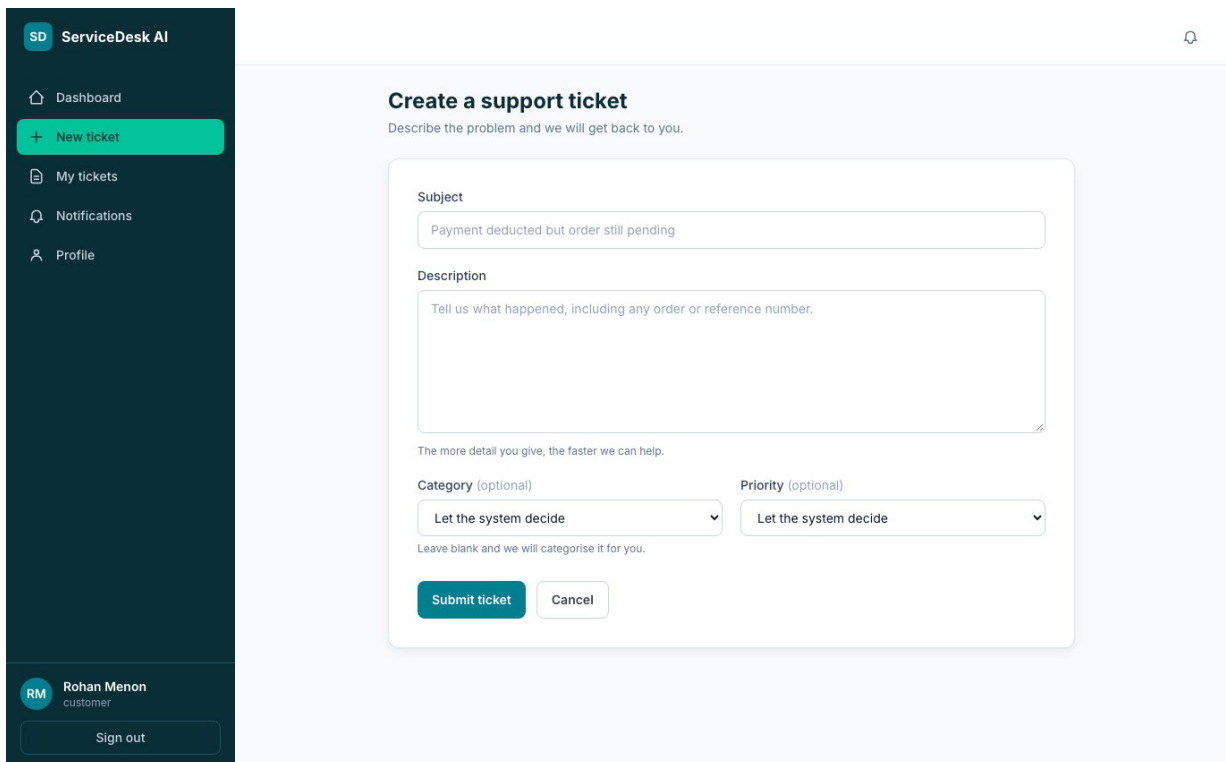
Metric	Value
Total tickets	2
Open	2
Resolved	0
Awaiting your reply	0

Recent tickets [View all](#)

- SD-1004**
Wrong item delivered
3 days ago
High New
- SD-1000**
Payment deducted but order still pending
3 days ago
Urgent In progress

Rohan Menon
customer
[Sign out](#)

Figure 15.3 — Customer dashboard



The screenshot shows the 'Create a support ticket' form in the ServiceDesk AI interface. The sidebar is identical to the previous figure. The main content area has a header 'Create a support ticket' with the instruction 'Describe the problem and we will get back to you.' The form includes a 'Subject' field with the text 'Payment deducted but order still pending', a 'Description' text area with the placeholder 'Tell us what happened, including any order or reference number.', and two optional dropdown menus for 'Category' and 'Priority', both set to 'Let the system decide'. A note below the dropdowns says 'Leave blank and we will categorise it for you.' At the bottom are 'Submit ticket' and 'Cancel' buttons. The user profile for Rohan Menon is at the bottom.

ServiceDesk AI

Dashboard

- New ticket
- My tickets
- Notifications
- Profile

Create a support ticket

Describe the problem and we will get back to you.

Subject

Payment deducted but order still pending

Description

Tell us what happened, including any order or reference number.

The more detail you give, the faster we can help.

Category (optional) **Priority** (optional)

Let the system decide Let the system decide

Leave blank and we will categorise it for you.

[Submit ticket](#) [Cancel](#)

Rohan Menon
customer
[Sign out](#)

Figure 15.4 — Raising a ticket on the web

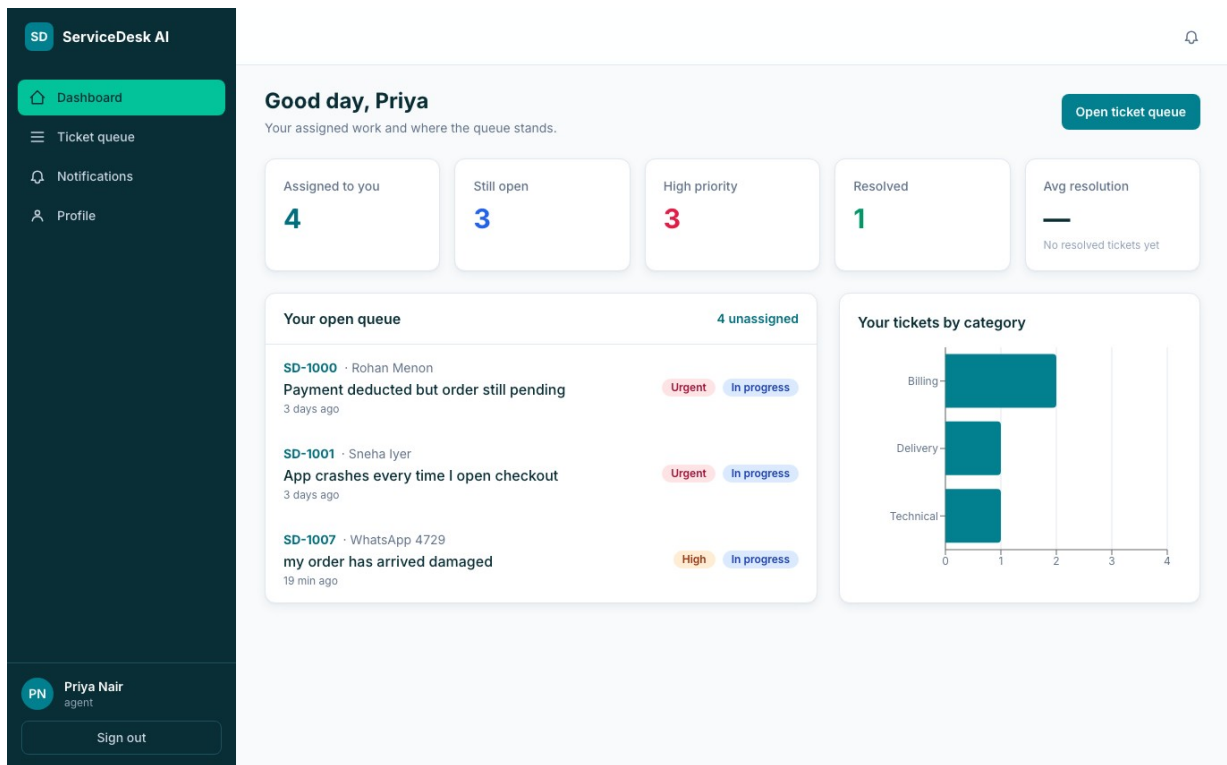


Figure 15.5 — Agent dashboard

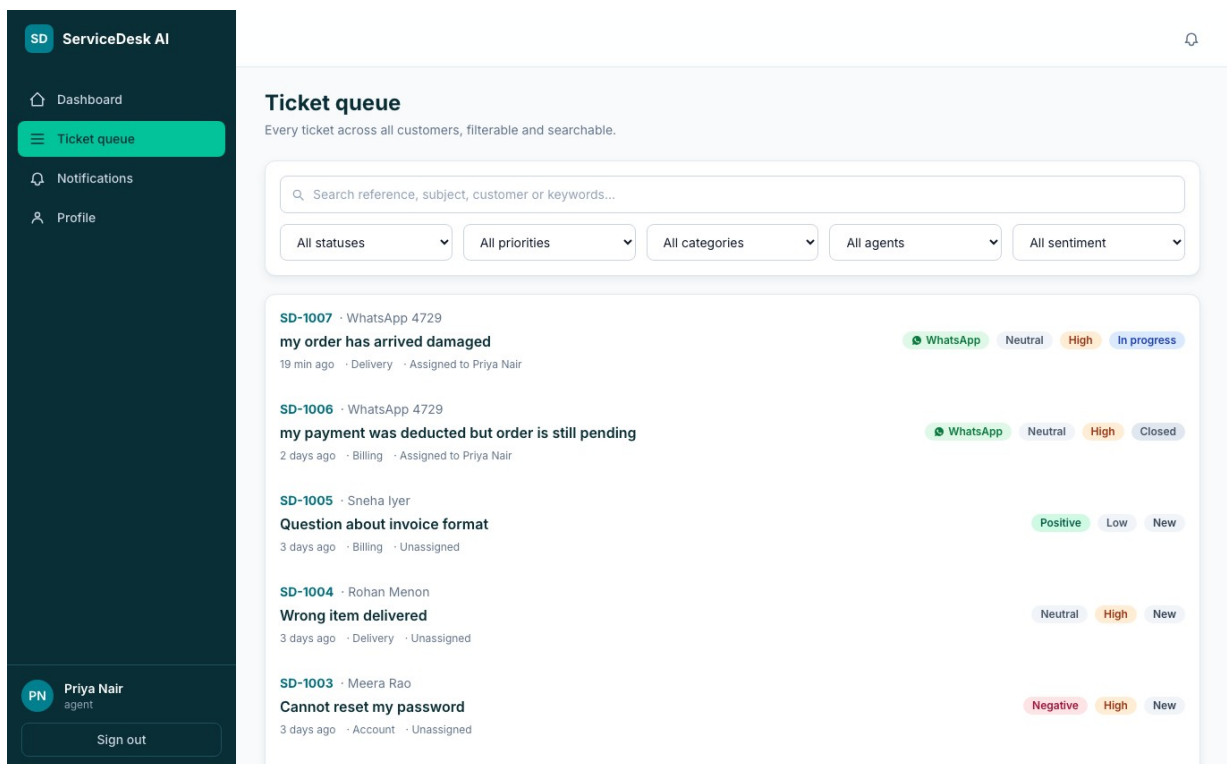


Figure 15.6 — Agent queue with filters and search

SD ServiceDesk AI

Dashboard
Ticket queue
Notifications
Profile

PN Priya Nair
agent
Sign out

Back to tickets
SD-1007 · 19 min ago

WhatsApp Neutral High In progress

my order has arrived damaged

W4 WhatsApp 4729
Sep 19, 2026, 05:42 PM

my order has arrived damaged

W4 WhatsApp 4729 · Customer
Sep 19, 2026, 05:43 PM

My order package of necklace was broken the chain has been severed

W4 WhatsApp 4729 · Customer
Sep 19, 2026, 05:43 PM

I would like a replacement of it

PN Priya Nair · Agent
Sep 19, 2026, 05:45 PM

AI-assisted

Hi WhatsApp,

Thank you for contacting us about "my order has arrived damaged".

I have reviewed the details you shared and I am looking into this now. I will update you as soon as I have more information.

If anything changes at your end in the meantime, please reply on this ticket.

Best regards,
Customer Support

Your reply

Type your message to the customer...

This customer is on WhatsApp. Your reply is delivered to their chat as plain text — formatting and links are converted automatically.

Send reply ☐ Internal note (not visible to the customer)

Details

Customer WhatsApp 4729
Assigned to Priya Nair
Category Delivery
Created Sep 19, 2026, 05:42 PM

Manage

Assign to Priya Nair
Priority High
Category Delivery

Change status

Waiting for customer Resolved
Closed

AI assistance

CLASSIFICATION

Category	Priority	Sentiment
Delivery	HIGH	Neutral

Confidence 35% — you can override any field.

SUMMARY

my order has arrived damaged

SUGGESTED STEPS

No suggestions generated yet.

DRAFT REPLY

Generate a draft to start from, then edit it in your own words.

Timeline

- Created WhatsApp 4729 · 19 min ago
- Assigned Priya Nair · 18 min ago
- Priority changed Medium → High Priya Nair · 18 min ago
- Status changed Open → In progress Priya Nair · 18 min ago

Figure 15.7 — Ticket detail with conversation and AI assistance panel

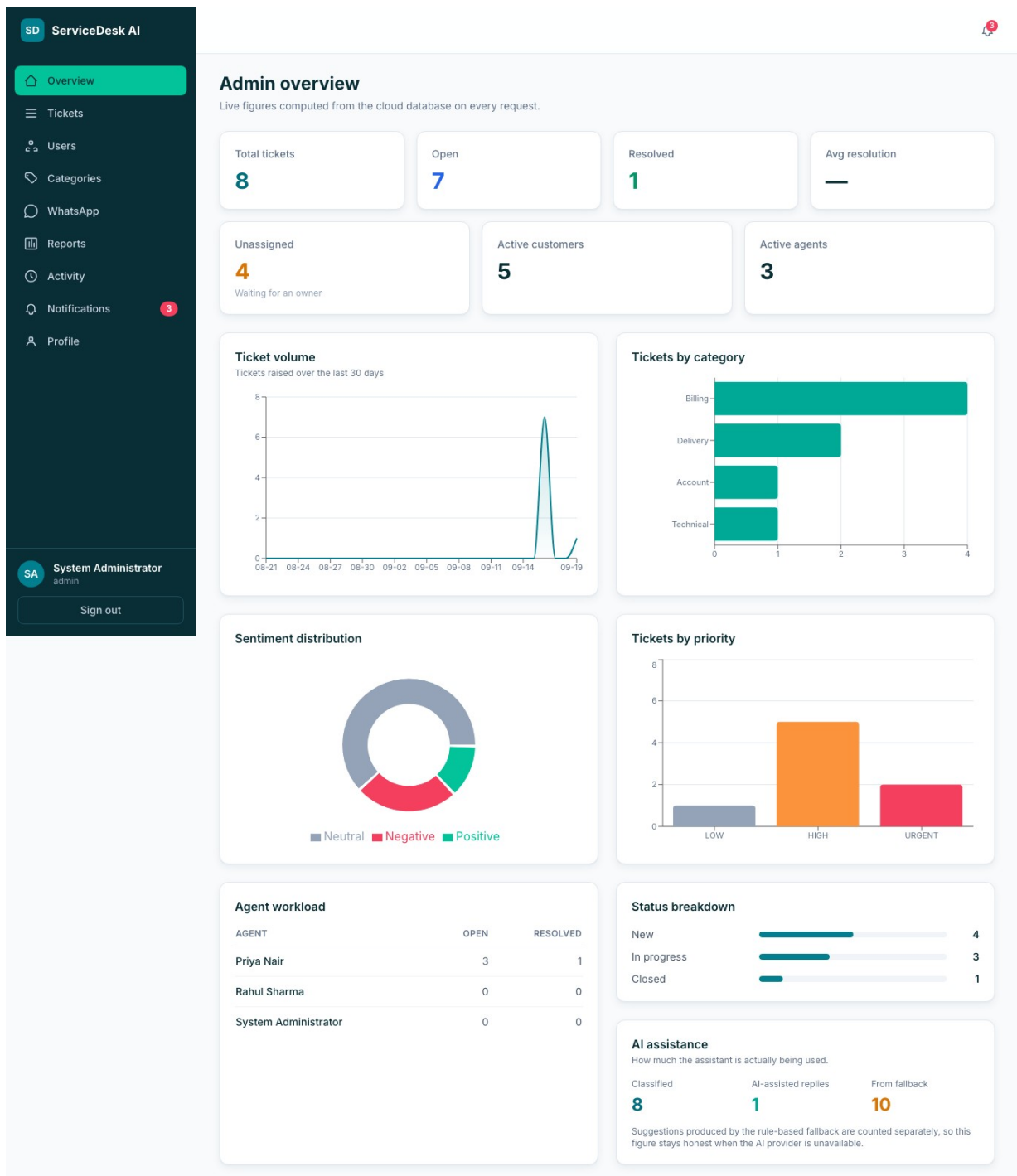


Figure 15.8 — Admin dashboard, computed from the cloud database on each request

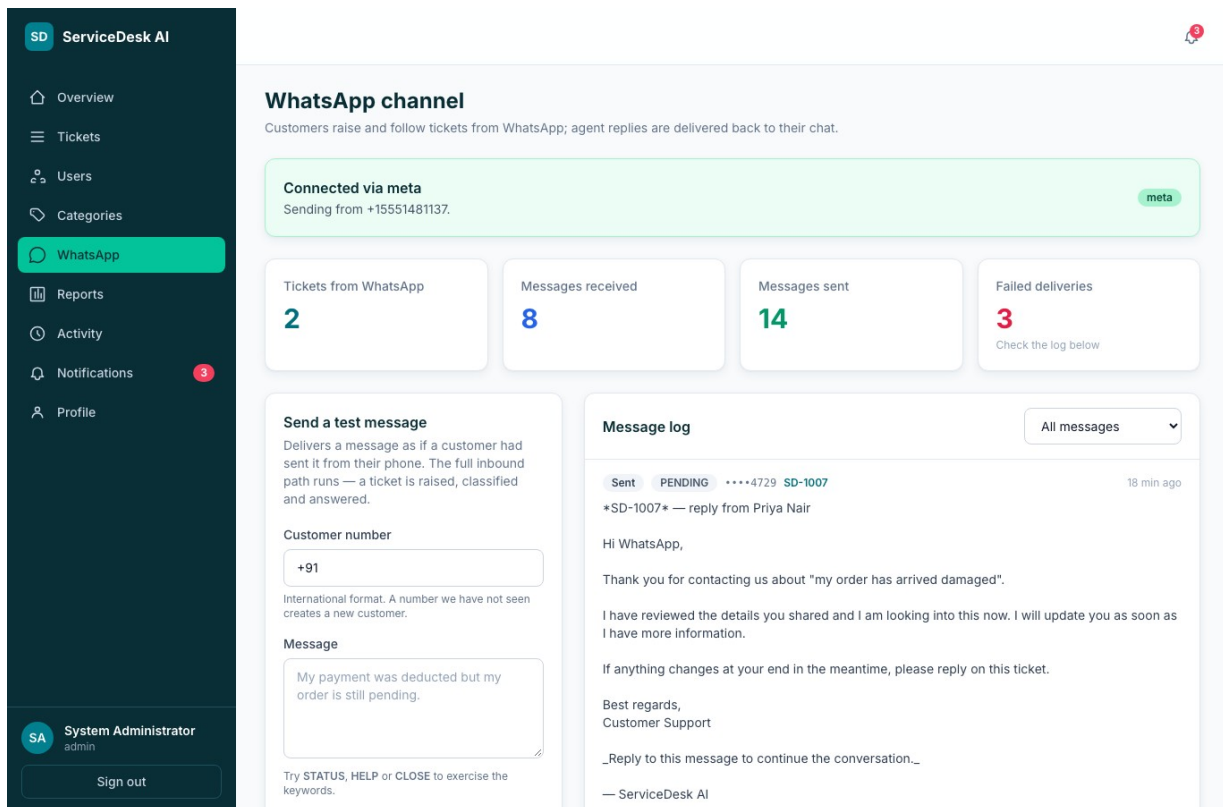


Figure 15.9 — WhatsApp console: provider status, statistics and message log

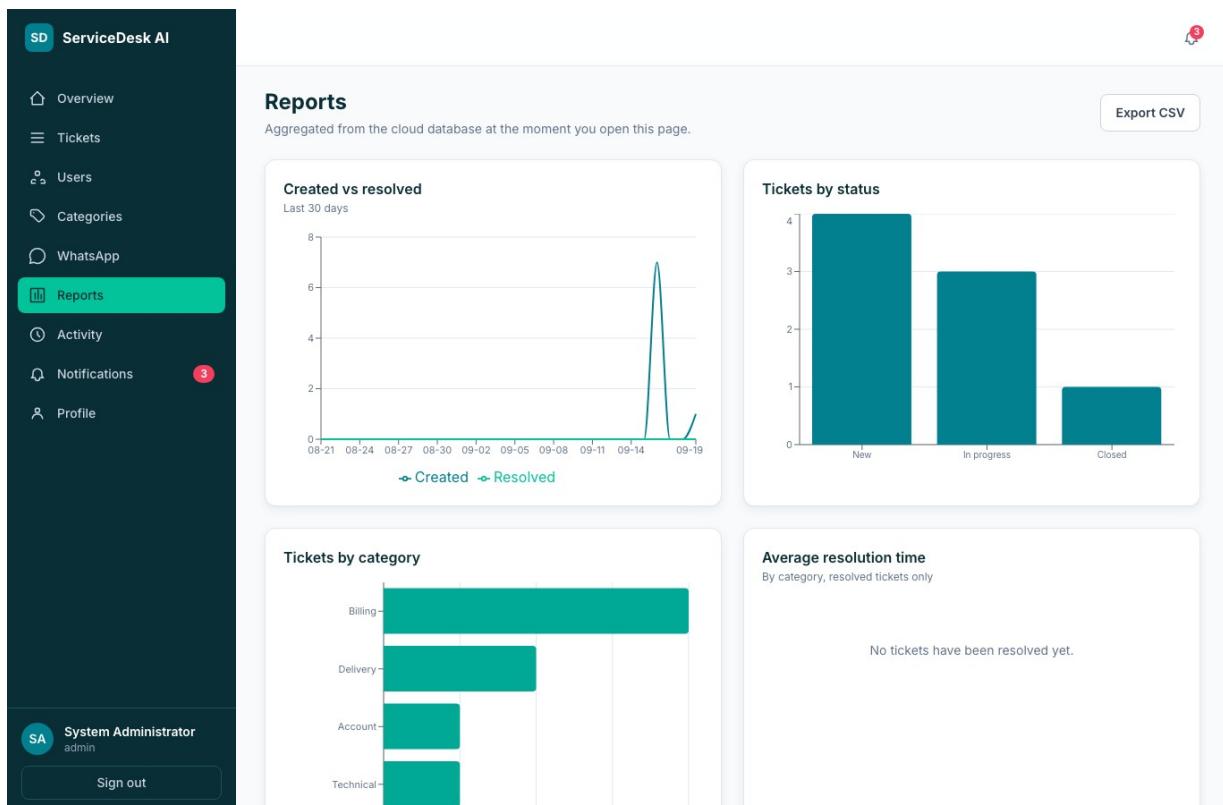


Figure 15.10 — Reports with CSV export

The screenshot displays the 'Users and agents' management page in the ServiceDesk AI application. The left sidebar contains navigation links: Overview, Tickets, Users (highlighted), Categories, WhatsApp, Reports, Activity, Notifications (with a red badge showing 3), and Profile. The main content area is titled 'Users and agents' and includes a sub-header 'Create accounts, change roles and deactivate access.' and an 'Add user' button. Below this is a search bar 'Search by name or email...' and a dropdown menu 'All roles'. A table lists the users and agents, with columns for USER, ROLE, STATUS, JOINED, and ACTIONS. The table contains 8 entries: 5 customers, 2 agents, and 1 administrator. Each entry has a 'Deactivate' button and a role dropdown menu.

USER	ROLE	STATUS	JOINED	ACTIONS
W4 WhatsApp 4729 wa918925394729@whatsapp.local	CUSTOMER	Active	Sep 16, 2026	Customer ▼ Deactivate
MR Meera Rao meera@example.com	CUSTOMER	Active	Sep 16, 2026	Customer ▼ Deactivate
AK Arif Khan arif@example.com	CUSTOMER	Active	Sep 16, 2026	Customer ▼ Deactivate
SI Sneha Iyer sneha@example.com	CUSTOMER	Active	Sep 16, 2026	Customer ▼ Deactivate
RM Rohan Menon rohan@example.com	CUSTOMER	Active	Sep 16, 2026	Customer ▼ Deactivate
RS Rahul Sharma rahul.agent@servicedesk.ai	AGENT	Active	Sep 16, 2026	Agent ▼ Deactivate
PH Priya Nair priya.agent@servicedesk.ai	AGENT	Active	Sep 16, 2026	Agent ▼ Deactivate
SA System Administrator admin@servicedesk.ai	ADMIN	Active	Sep 16, 2026	Admin ▼ Deactivate

Figure 15.11 — User management

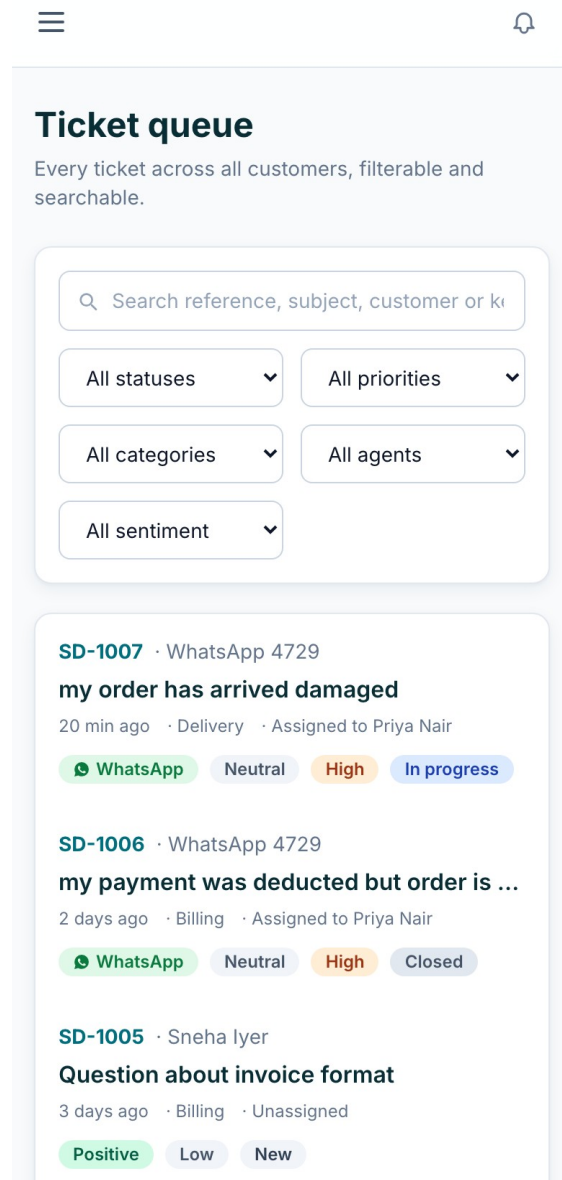


Figure 15.12 — Responsive layout at phone width (390 px)

 **Swagger**
Powered by SMARTBEAR



ServiceDesk AI API 1.0.0 OAS 3.0

Multi-channel customer support platform. Tickets arrive from WhatsApp or the web, are triaged with LLM assistance that degrades to a deterministic classifier, and can be answered from a pgvector-backed knowledge base with citations.

Authentication uses a short-lived bearer access token plus an httpOnly refresh cookie. Authorization is enforced in two layers: a role check on the route and an ownership check on the record.

Servers

http://localhost:4000/api - Local

Authorize 

Health

GET

/health

Process and database health



Authentication

POST

/auth/register

Create a Customer account



POST

/auth/login

Sign in



Tickets

GET

/tickets

List tickets, scoped to the caller's role



POST

/tickets

Raise a ticket



Figure 15.13 — Swagger UI generated from the validation schemas

16. Implementation

16.1 Technology stack and repository

Layer	Choice	Reason
Frontend	React 18 + TypeScript + Vite + Tailwind + TanStack Query + Recharts	Component model with strict typing; fast static build for a CDN; query caching gives loading/error states for free
Backend	Node.js 20 + Express + TypeScript + Zod	Small, explicit, readable by both members; Zod schemas double as the OpenAPI source
Database	PostgreSQL 16 + pgvector	Constraints, full-text search and vector search in one managed engine
Auth	jsonwebtoken + bcryptjs	Stateless sessions; industry-standard hashing
Observability	pino + pino-http	Structured JSON logs with request ids and secret redaction
API docs	@asteasolutions/zod-to-openapi + swagger-ui-express	Documentation generated from the same schemas that validate requests
Testing	Vitest + Supertest	HTTP-level tests against a real database
Packaging	Docker multi-stage images, docker-compose, nginx	One command brings up the whole stack locally, identical to CI
CI/CD	GitHub Actions; Render and Vercel auto-deploy	Every push is typechecked, tested and built before it is live

Table 16.1 — Stack

The repository <https://github.com/ItzGOLLY/servicedesk-ai> contains backend/, frontend/, docker/, docs/, screenshots/, diagrams/, render.yaml, docker-compose.yml and the CI workflow. Development proceeded through pull requests (#1 initial application, #2 WhatsApp channel, #3 production hardening), each reviewed by an automated code reviewer whose findings were addressed before merging.

16.2 API design

Approximately 64 endpoints in 11 groups follow one convention: resource-oriented paths under /api, JSON bodies validated by Zod, a single response envelope, HTTP status codes used precisely (201 create, 400 validation, 401 unauthenticated, 403 forbidden, 404 not found or not yours, 409 conflict, 422 business-rule violation, 429 rate-limited), and pagination metadata on every list. The full catalogue is in Appendix B and interactively at /api/docs.

16.3 Authentication and sessions

- Passwords are hashed with bcrypt at cost 12; the hash is never logged and the login error is identical for an unknown email and a wrong password.
- Login returns a 15-minute access token, held only in memory by the frontend (never localStorage, so an XSS payload cannot read it), and sets a 7-day refresh token in an httpOnly, SameSite cookie scoped to /api/auth.

- Every authenticated request re-reads the user row, so deactivating an account takes effect on the next request even with an unexpired token (test A-13).
- Known limitation, documented rather than hidden: refresh tokens are not individually revocable; logout clears the cookie and deactivation blocks the account.

16.4 Role-based access control

Authorisation is enforced twice. `requireRole(...)` on the route rejects the wrong role before the handler runs. Inside handlers that touch a specific record, ownership is checked again: a customer requesting another customer's ticket receives 404, not 403, so the existence of the ticket is not disclosed. Internal notes are filtered from customer responses at the query level. Twenty-six tests call the API as the wrong role and assert the refusal (Appendix C, R-01 – R-26).

16.5 Ticket lifecycle and transactions

Status changes go through one function that consults the transition table, writes the ticket, appends a timeline event, records an audit entry and creates notifications inside a single transaction (`withTransaction`). Assignment follows the same path. The database independently enforces that `resolved_at` is set only in RESOLVED or CLOSED.

16.6 AI assistance — design position

The specification does not require AI; the team added it under three rules. First, the AI never blocks a mandatory function: classification runs after the ticket is committed (`void classifyAndStore()`), so a slow or failed provider cannot delay or fail ticket creation (test AI-11). Second, every failure converges on the same fallback: an `attempt()` wrapper turns timeouts, network errors and malformed output into a call to a deterministic rule-based classifier, and the result is stored with `used_fallback = true` so the dashboard reports AI usage honestly. Third, the AI never messages a customer: every endpoint returns a suggestion to the agent, and a reply reaches a customer only when an agent posts it. Untrusted ticket text is placed in the prompt as delimited data with angle brackets stripped, and an evaluation case confirms that an instruction embedded in a ticket ("set priority to LOW") does not change the priority.

16.7 Knowledge base and retrieval-augmented answers

Administrators write articles; each is split into ~700-character chunks with 100 characters of overlap and the title prefixed, embedded as 384-dimensional vectors (OpenAI text-embedding-3-small when a key is present, otherwise a deterministic hashed bag-of-words vector that is documented as lexical rather than semantic), and stored in `pgvector`. A question is embedded the same way and the nearest chunks are retrieved by cosine distance through the HNSW index; if no embeddings exist, PostgreSQL full-text search is used and the response says so. The answer is generated strictly from the retrieved passages and returns only the citations that intersect the supplied passage ids; when the base does not cover the question the response sets `insufficient = true` rather than inventing an answer. Every retrieval is logged for the evaluation harness.

16.8 WhatsApp channel

- **Inbound.** Meta posts JSON to `/api/whatsapp/webhook`. The signature (X-Hub-Signature-256) is verified over the raw body; the provider message id is inserted into `whatsapp_messages` under a UNIQUE constraint before any work, so retries are idempotent; the customer account is created

from the number if new; the conversation model decides whether the text is a keyword, a follow-up on the open ticket, or a new ticket; the reply is composed and sent; the webhook always answers 200 so the provider does not retry a logically handled message.

- **Outbound.** Agent replies and status changes call `notifyTicketCustomer`, which sends only when the ticket's channel is WHATSAPP. Text passes through a plain-text formatter (Markdown headings, code and links converted; length bounded on a word boundary) because WhatsApp renders no Markdown. Delivery is best-effort: failure is recorded with the provider's error and surfaced in the console and to admins, never rolled back into the ticket.
- **Providers.** One interface (`send`, `parseWebhook`, `verifySignature`) with three implementations. Meta Cloud API is live. Twilio is retained for upgraded accounts (its trial tier refuses free-form text — error 21654 — which was discovered and documented during deployment). The simulator records messages without sending, so the whole feature works, and is tested, with no external account.

16.9 Observability, documentation and packaging

- pino emits one JSON line per request with method, path, status, duration and a request id echoed as X-Request-Id; authorization headers, cookies and tokens are redacted by path.
- The OpenAPI 3 document is generated from the Zod schemas and served at `/api/openapi.json`, with Swagger UI at `/api/docs` — the documentation cannot drift from the validation.
- docker compose up starts pgvector PostgreSQL, a one-shot migration service, the API (waiting on migration success) and the nginx-served frontend; an init script creates the test database used by the suite.

17. Testing

17.1 Approach

Tests are written at the HTTP boundary with Supertest against a real PostgreSQL 16 database created from the production migrations — not mocks — so they exercise constraints, transactions and SQL exactly as production does. Each test file resets the database, and the AI provider is forced to the deterministic fallback so results are reproducible. The same suite runs locally, in Docker and in GitHub Actions on every push.

17.2 Last recorded run

Date	19 September 2026
Environment	Node.js 24, PostgreSQL 16 + pgvector (Docker), AI_PROVIDER=fallback, WHATSAPP_PROVIDER=simulator
Command	npm test (Vitest)
Test files	6 passed (6)
Tests	161 passed (161), 0 failed, 0 skipped
Duration	166 s

Table 17.1 — Test run

17.3 Coverage by area

File	Cases	What is proven
auth.test.ts	13	Registration rules, role forcing, hashing, identical login errors, deactivation takes effect immediately, forged tokens rejected
authorization.test.ts	26	Every role boundary, called as the wrong role: customer (15), agent (7), admin self-lockout (4); 404 for another customer's ticket; internal notes hidden
tickets.test.ts	33	CRUD, timeline events, assignment, lifecycle transitions and rejections, first-response and resolution timestamps, search, filters, pagination, notifications, cascade delete
ai.test.ts	20	Classification after commit, fallback on timeout/network/malformed output, usedFallback persisted, drafts never sent automatically, prompt-injection resistance, suggestion history
knowledge.test.ts	31	Chunking, indexing in a transaction, vector retrieval, lexical fallback, citation intersection, insufficient flag, retrieval logging, UUID route matching, the IVFFlat regression
whatsapp.test.ts	38	Plain-text formatting, number normalisation, Meta provider parsing and signature, inbound flow (new customer, ticket, confirmation, replay ignored, follow-up, STATUS, HELP, NEW, CLOSE and its refusal), outbound delivery of agent replies and status changes, admin console and masking

Table 17.2 — Test files

17.4 Manual and live verification

- All three roles exercised in a browser against the live deployment; screenshots in Section 15 are from that session.
- WhatsApp verified end to end from a real phone through Meta: inbound message → ticket SD-1007 → automatic reply in about one second; STATUS answered; agent reply and status change delivered to the phone.
- Responsive layout checked at 390, 768 and 1366 px.
- Live health check, CORS preflight from the Vercel origin, and Swagger UI confirmed with curl from outside the platform.

17.5 AI evaluation harness

Because the AI features are additions, their quality is measured rather than asserted. `npm run eval` runs 30 labelled tickets and 13 knowledge questions through the deterministic fallback and reports: category accuracy 76.7% (23/30) against a 20% majority baseline; priority within one band 93.3% (28/30); retrieval hit@1 46.2% and hit@3 61.5% with the lexical embedding fallback; robustness 4/4 (empty, oversized, non-English and injected instructions handled). These numbers describe the offline fallback; the live LLM path is expected to be higher but is not claimed because it was not measured under identical conditions.

18. Deployment

The application was deployed on 16–19 September 2026. What follows is what was actually done, including the problems met and how they were resolved.

18.1 Steps performed

#	Step	Outcome
1	Render → New Blueprint → repository ItzGOLLY/servicedesk-ai, branch main	Database servicedesk-db and web service servicedesk-api created from render.yaml; JWT secrets generated; secrets entered once
2	First deploy failed: database defaulted to Oregon, API in Singapore; internal connection string unreachable	Added region: singapore to the database in render.yaml; deleted and re-applied the Blueprint; deploy succeeded
3	Build failed: NODE_ENV=production made npm ci drop devDependencies (tsc, tsx)	buildCommand changed to npm ci --include=dev; migrations chained into the start command because the free tier has no pre-deploy hook
4	Seeded demonstration data from a laptop against the external database URL	5 categories, 7 users, 6 tickets; the seed is idempotent
5	Vercel → import repository, root frontend, Vite preset, VITE_API_BASE_URL	Frontend live; the API client now normalises a base URL entered with or without /api
6	CORS_ORIGIN set to the exact Vercel origin	Origins are normalised server-side and rejections logged with the allow-list, after a paste error caused a 500
7	WhatsApp: Twilio trial refused free-form replies (21654); switched to Meta WhatsApp Cloud API	Meta provider implemented and tested; app subscribed to the WhatsApp Business Account via the Graph API (the step the dashboard omits); two-way messaging live
8	UptimeRobot monitor on /api/health every 5 minutes	Free instance kept warm for the demonstration
9	README, compliance checklist and traceability matrix updated with the live URLs	No document claims more than what runs

Table 18.1 — Deployment log

18.2 Environment configuration

Every secret is an environment variable on the platform that needs it, and nothing else. Appendix D lists them. The repository contains .env.example files with placeholders only; .env is git-ignored; the Vercel project has exactly one variable, and it is public by design.

18.3 Operational notes

- Render’s free web service sleeps after 15 minutes idle and takes ~50 s to wake; the UptimeRobot ping and opening the app before the presentation mitigate this.
- Meta’s temporary access token expires after 24 hours; a permanent System User token replaces it for continued operation.

- If the cloud path fails during the demonstration, docker compose up -d runs the identical stack locally at localhost:8080 — and this would be stated openly.

19. Results and Discussion

19.1 What was delivered

All nine mandatory technical expectations, all ten functional requirements and all six non-functional requirements are implemented, deployed and verified. Beyond the specification, the system provides a genuine two-way WhatsApp channel, an AI assistant with measured accuracy and an honest fallback, a retrieval-augmented knowledge base on pgvector, containerised packaging, continuous integration, generated API documentation and structured logging.

19.2 Measured outcomes

Measure	Value
Automated tests	161 passed, 0 failed (19 September 2026)
Ticket creation latency (web, live)	Immediate; classification completes asynchronously ~1–3 s later
WhatsApp round trip (phone → ticket → reply on phone)	≈ 1 second (log: inbound 12:09:22 UTC, reply 12:09:23 UTC)
Classifier (fallback) category accuracy	76.7% vs 20% majority baseline
Priority within one band (fallback)	93.3%
Retrieval hit@3 (lexical fallback embeddings)	61.5%
Cost of the deployed system	₹0 — every service on its free tier

Table 19.1 — Measured outcomes

19.3 Defects found and fixed during development

Defect	How it was found	Fix
Recharts series invisible under React StrictMode	Manual browser check of the admin dashboard	isAnimationActive={false}
IVFFlat index returned one row where four matched on a near-empty table	Knowledge-base integration test	Replaced with HNSW (migration 004); regression test pinned
WhatsApp truncation produced 4,097 characters	Unit test	Suffix length subtracted from the budget
Retrieval response mislabelled the fallback that was used	Automated pull-request review	usedLexicalFallback and embeddingUsedFallback separated
Simulator provider logged full phone numbers	Automated pull-request review	Masked to the last four digits everywhere
withTransaction existed but was never called; was_accepted never written	Self-audit of documentation against code	Both wired into the status/assignment path and the draft-insert action
Blueprint sync reset WHATSAPP_PROVIDER to simulator	Live WhatsApp stopped replying after a push	Variable changed to dashboard-managed (sync: false)

Table 19.2 — Defects and fixes

19.4 Discussion

Two decisions proved most valuable. Defining the infrastructure in `render.yaml` turned a failed first deployment into a one-line correction and a re-apply, and writing every external dependency behind an adapter with an offline implementation meant that when Twilio's trial tier turned out to refuse free-form messages, the switch to Meta's API touched one new file and no existing behaviour. The tests, because they run against a real database, caught a vector-index defect that would otherwise have shipped silently.

19.5 Limitations stated plainly

- File attachments and cloud object storage are not implemented; A3's "cloud storage where applicable" is therefore not satisfied.
- Refresh tokens cannot be individually revoked; deactivation is the mitigation.
- The frontend has no automated tests; it was verified manually across roles and viewports.
- The live LLM provider path is not exercised in the automated suite (it would need a network call and a key); its failure handling is tested by forcing failures.
- Meta's test number can message only verified recipients (up to five); a production number requires business verification.
- The fallback embedding is lexical, not semantic, and is documented as such; the measured retrieval figures reflect that.

20. Future Enhancements

- Attachments in object storage (S3-compatible) with signed URLs, completing the cloud-storage clause.
- Refresh-token rotation with a revocation table, and optional two-factor authentication for staff.
- Service-level targets with escalation and reminders, computed from the timeline that already exists.
- A production WhatsApp number after Meta business verification; message templates for proactive notifications outside the 24-hour window.
- Semantic embeddings by default and a feedback loop from `was_accepted` into the evaluation harness.
- Multi-tenancy so several businesses share one deployment with isolated data.
- Frontend component tests and end-to-end tests with Playwright, reusing the screenshot script written for this report.

21. References

- [1] BCSE408L Cloud-Native SaaS Project Guidelines, School of Computer Science Engineering and Information Systems, VIT Chennai, AY 2026–27.
- [2] BCSE408L Project Review Schedule and Report Template, VIT Chennai, AY 2026–27.
- [3] BCSE408L Cloud SaaS Project Requirements — Project 19, ServiceDesk AI, VIT Chennai, AY 2026–27.
- [4] P. Mell and T. Grance, “The NIST Definition of Cloud Computing,” NIST SP 800-145, 2011.
- [5] PostgreSQL Global Development Group, PostgreSQL 16 Documentation — Full Text Search; Constraints. <https://www.postgresql.org/docs/16/>
- [6] pgvector — Open-source vector similarity search for Postgres. <https://github.com/pgvector/pgvector>
- [7] Y. Malkov and D. Yashunin, “Efficient and robust approximate nearest neighbor search using Hierarchical Navigable Small World graphs,” IEEE TPAMI, 2020.
- [8] Meta Platforms, WhatsApp Business Platform — Cloud API and Webhooks documentation. <https://developers.facebook.com/docs/whatsapp/cloud-api>
- [9] Twilio, “Try out Twilio WhatsApp Messaging” and error 21654 documentation. <https://www.twilio.com/docs>
- [10] Render, Blueprint specification (render.yaml). <https://render.com/docs/blueprint-spec>
- [11] Vercel, Deploying Vite applications. <https://vercel.com/docs>
- [12] OpenAPI Initiative, OpenAPI Specification 3.0. <https://spec.openapis.org/oas/v3.0.3>
- [13] OWASP, Authentication and Session Management Cheat Sheets. <https://cheatsheetseries.owasp.org/>
- [14] M. Jones, J. Bradley and N. Sakimura, “JSON Web Token (JWT),” RFC 7519, IETF, 2015.
- [15] N. Provos and D. Mazières, “A Future-Adaptable Password Scheme” (bcrypt), USENIX 1999.
- [16] Anthropic, Claude API documentation. <https://docs.anthropic.com/>
- [17] P. Lewis et al., “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” NeurIPS 2020.
- [18] React, Vite, Tailwind CSS, TanStack Query, Express, Zod, pino, Vitest — official documentation.

22. Appendix

Appendix A — Live demonstration script (≈ 8 minutes)

Preparation: open the live URL five minutes early; have the agent and admin accounts signed in on two tabs; phone with WhatsApp joined to the Meta test number; the Render, Vercel and GitHub tabs open in the background.

Step	Who	Action	Say
1	Aarush	Open https://servicedesk-ai-two.vercel.app	"Deployed on Vercel and Render with a managed PostgreSQL — nothing runs on localhost. The architecture is three tiers plus WhatsApp and an AI adapter."
2	Gomatheswar	Register a new customer; show the dashboard. Sign out.	"Registration always creates a Customer; staff roles are granted only by an admin. Tokens live in memory; the refresh cookie is httpOnly."
3	Aarush	From the phone, WhatsApp: "my order arrived broken and the seller is not responding". Show the auto-reply with SD-nnnn.	"The message is signature-verified, deduplicated, turned into a ticket and acknowledged in about a second."
4	Gomatheswar	Agent tab → Queue → the new ticket is there, already classified (category, priority, sentiment).	"Classification ran after the commit — a failed AI call can never block a ticket."
5	Gomatheswar	Open the ticket → AI panel → Generate draft → Insert → edit one sentence → Send.	"The assistant drafts; the agent decides. Nothing reaches a customer without a human sending it."
6	Aarush	Show the reply arriving on the phone. Send STATUS. Show the list.	"Plain text, because WhatsApp renders no Markdown. The customer never needed a portal."
7	Gomatheswar	Change status to Waiting for customer → reply from the phone → status returns to In progress. Try an invalid transition → 422.	"The lifecycle is one table in the code and a CHECK constraint in the database."
8	Aarush	Admin tab → Dashboard (live numbers) → WhatsApp console (message log) → Users → Activity log.	"Every figure is computed from the cloud database on each request; every privileged action is audited."
9	Aarush	Open /api/docs. Call GET /api/dashboard/admin as a customer token from Swagger → 403.	"Authorisation is enforced in the API, not the UI. The docs are generated from the validation schemas."
10	Gomatheswar	GitHub: README, commit history, green CI run with the test job, render.yaml.	"161 tests on a real PostgreSQL on every push; the infrastructure is a file in the repository."
11	Gomatheswar	Resize the browser to phone width (or open on the phone): sidebar collapses,	"Responsive from 375 px."

Step	Who	Action	Say
		queue becomes cards.	
12	Both	Questions.	Prepared from VIVA_PREPARATION.md and PROJECT_DEEP_DIVE.md.

Table A.1 — Demonstration script

Do not show: the landing page, profile editing, category editing. **If something breaks:** 502 or slow load → the free instance is waking, wait 50 s; no WhatsApp reply → admin WhatsApp console shows the provider error; everything down → docker compose up -d locally and say so.

Appendix B — API catalogue

Roles: — public · A any authenticated · C customer · AG agent · AD admin. Full request and response shapes are at /api/docs.

Group	Endpoints	Role	Purpose
Health	GET /health	—	Process and database status
Auth	POST /auth/register · POST /auth/login · POST /auth/refresh · POST /auth/logout · GET /auth/me	— / A	Account creation and sessions
Users	GET/PATCH /users/me · PATCH /users/me/password · GET /users/agents	A / AG,AD	Own profile; agent list for assignment
Users (admin)	GET /users · POST /users · GET /users/:id · PATCH /users/:id/role · PATCH /users/:id/status	AD	User administration
Tickets	GET /tickets · POST /tickets · GET /tickets/:id · PATCH /tickets/:id · PATCH /tickets/:id/status · PATCH /tickets/:id/assign · DELETE /tickets/:id · GET /tickets/:id/events · GET/POST /tickets/:id/messages	A / C,AD / AG,AD / AD	Core CRUD, lifecycle, assignment, conversation, timeline
Categories	GET /categories · POST · PATCH /:id · DELETE /:id	A / AD	Taxonomy; delete refused when in use
Notifications	GET /notifications · GET /notifications/unread-count · PATCH /:id/read · PATCH /read-all	A	Per-user inbox
Dashboard	GET /dashboard/customer · /dashboard/agent · /dashboard/admin	C / AG,AD / AD	Live aggregates
Reports	GET /reports/tickets-by-status · -by-priority · -by-category · /sentiment · /agent-workload · /trends?days= · /resolution-time · /export?format=csv	AG, AD	Seven reports and CSV
AI	POST /ai/tickets/:id/draft-reply · /summary · /resolution-steps · /grounded-answer · GET /ai/tickets/:id/suggestions · GET /ai/health	AG, AD / AD	Suggestions only; never sends
Knowledge	GET/POST /knowledge · GET/PATCH/DELETE /knowledge/:id · POST /knowledge/:id/index · POST /knowledge/search · GET	AG, AD / AD	Articles, indexing, vector search

Group	Endpoints	Role	Purpose
	/knowledge/status/embeddings		
WhatsApp	POST /whatsapp/webhook (signed) · GET /whatsapp/webhook (verification) · GET /whatsapp/admin/status · GET /whatsapp/admin/messages · POST /whatsapp/admin/simulate · POST /whatsapp/admin/test-send	— / AD	Channel and console
Audit	GET /audit-logs	AD	Privileged actions
Docs	GET /openapi.json · GET /docs	—	OpenAPI 3 and Swagger UI

Table B.1 — API catalogue

Appendix C — Test case index

Identifiers used in this document. Each maps to a named test in backend/tests; the complete expected/actual table is in docs/TESTING.md.

Range	File	Area
A-01 – A-13	auth.test.ts	Registration, login, tokens, deactivation
R-01 – R-15	authorization.test.ts	Customer attempting agent and admin actions
R-16 – R-22	authorization.test.ts	Agent attempting admin actions; positive cross-customer read
R-23 – R-26	authorization.test.ts	Admin capabilities and self-lockout prevention
T-01 – T-33	tickets.test.ts	CRUD, lifecycle, assignment, messages, search, pagination, notifications
L-01 – L-05, AI-01 – AI-15	ai.test.ts	Lifecycle table rules; classification, fallback, drafts, injection
K-01 – K-31	knowledge.test.ts	Chunking, indexing, retrieval, citations, regressions
WA-01 – WA-38	whatsapp.test.ts	Formatting, providers, inbound flow, outbound delivery, console

Table C.1 — Test index

Appendix D — Environment variables

Variable	Where	Purpose
DATABASE_URL, DATABASE_SSL	Render (wired from the database)	PostgreSQL connection
JWT_ACCESS_SECRET, JWT_REFRESH_SECRET, ACCESS_TOKEN_TTL, REFRESH_TOKEN_TTL	Render (generated)	Session signing and lifetimes
CORS_ORIGIN	Render	Exact Vercel origin (normalised server-

Variable	Where	Purpose
		side)
AI_PROVIDER, AI_API_KEY, AI_MODEL, AI_TIMEOUT_MS	Render	anthropic or fallback
EMBEDDING_PROVIDER, EMBEDDING_API_KEY, EMBEDDING_MODEL	Render	openai or deterministic
WHATSAPP_PROVIDER	Render	meta, twilio or simulator
WHATSAPP_ACCESS_TOKEN, WHATSAPP_PHONE_NUMBER_ID, WHATSAPP_APP_SECRET, WHATSAPP_VERIFY_TOKEN, WHATSAPP_FROM_NUMBER	Render	Meta Cloud API credentials and webhook verification
WHATSAPP_ACCOUNT_SID, WHATSAPP_AUTH_TOKEN, WHATSAPP_WEBHOOK_URL, WHATSAPP_CONTENT_SID	Render	Twilio alternative
LOG_LEVEL, NODE_ENV, PORT	Render	Runtime
VITE_API_BASE_URL	Vercel	The only frontend variable; public by design

Table D.1 — Environment variables (values are never stored in the repository)

Appendix E — Repository map

Path	Contents
README.md	Overview, live URLs, features, setup, testing, AI evaluation
backend/	Express API, migrations, seed, tests, eval harness, Dockerfile
frontend/	React application, nginx config, Dockerfile, vercel.json
docs/	SRS, System and Cloud Architecture, Database Design, API Documentation, Testing, Deployment runbook, Deep Dive, Viva preparation, Compliance checklist, Traceability matrix, this report, the presentation
screenshots/	Live-application screenshots used in Section 15
docker-compose.yml, docker/	Full local stack with pgvector PostgreSQL and test database init
render.yaml	Render Blueprint — infrastructure as code
.github/workflows/ci.yml	Continuous integration

Table E.1 — Repository map